INTELIGENCIA computacional

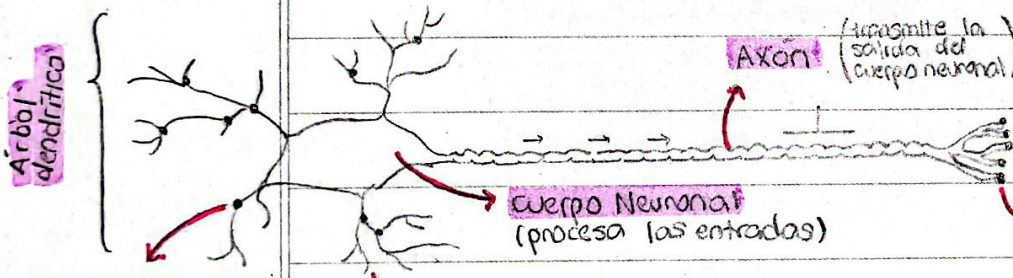
no tenemos tanta capacidad o eficiencia de cómputo como compu

Redes Neuronales

buscan tener capacidades del ser humano. no tienen creatividad y razonamiento pero tienen:

- **No linealidad:** Pequeño cambio en entrada → cambio grande en salida
- **Paralelismo:** Neuronas sencillas q' concurren pl resolver problemas complejos.
- **Aprendizaje:** Pueden aprender solas, no tengo q' programar línea a línea.
- **Generalización:** Puede resolver problemas q' nunca vio exactamente igual.
- **Adaptabilidad:** Adaptarse a nuevas situaciones a medida q' van funcionando.
- **Robustez:** Funciona bien ante datos con ruido o datos faltantes.

Neurona Biológica



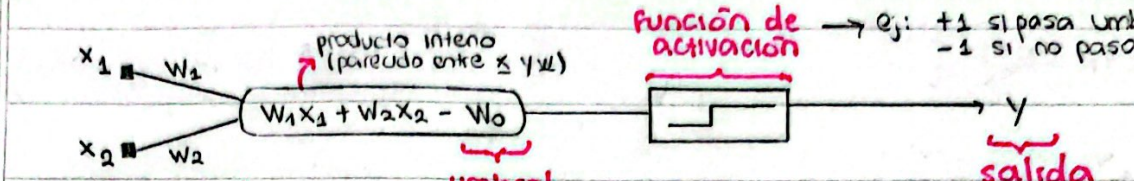
Éxito

Scanned with CamScanner

PERCEPTRÓN — modelo simplificado de la neurona biológica

- **SIMPLE:**

es una red estática, entra la entrada, sale la salida y listo. aprendizaje supervisado.



→ **Entrenamiento:**

1. Inicialización al azar los pesos.
2. Muestro ejemplos con respuestas correctas yd
   - ↳ si $y(n) = y_d(n)$, no cambio { principio de mínima perturbación
   - ↳ si $y(n) \neq y_d(n)$, actualizo los pesos en el sentido opuesto al que le erré { penalización
3. Vuelvo a 2 hasta satisfacer criterio de parada o número máximo de épocas.

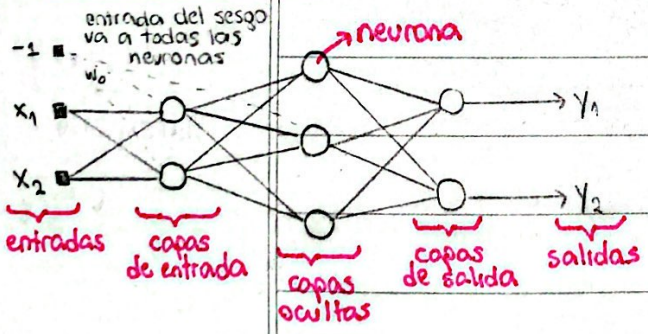
Fórmula generalizada:

$$\underline{w}[n+1] = \underline{w}[n] + \gamma[y_d[n] - y[n]] \cdot \underline{x}[n]$$

γ = velocidad de aprendizaje (qué tanto aprende de lo nuevo y olvida lo que aprendió antes.)

lo mismo pero con sustento numérico

- **MULTICAPAS:** red no estática, aprendizaje supervisado.



con dos capas puedo hacer lo que quiera pero son más neuronas que tres.

Método del gradiente

$$\underline{w}(n+1) = \underline{w}(n) - \mu \nabla_w \xi(\underline{w}(n))$$

gradiente del error

$$e_n^2 = [y_{d_n} - y_n]^2 = [y_{d_n} - \langle \underline{w}_n, \underline{x}_n \rangle]^2$$

$$\nabla_w e_n^2 = 2[y_{d_n} - \langle \underline{w}_n, \underline{x}_n \rangle] \cdot (-\underline{x}_n)$$

$$\nabla_w e_n^2 = 2 e_n (-\underline{x}_n)$$

$$\underline{w}(n+1) = \underline{w}(n) - 2\mu e(n) \cdot (-\underline{x}_n)$$

→ **Entrenamiento:**

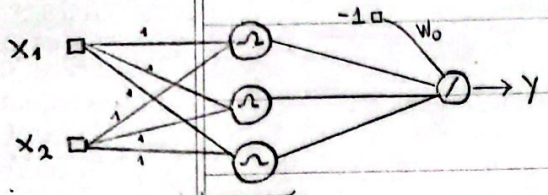
1. Inicializo al azar los pesos.
2. Propagación hacia adelante de las entradas (salidas de una capa son entradas de las próximas capas)
3. Propagación hacia atrás del error para actualizar los pesos → Aplicación del gradiente
4. Ajuste de pesos con los delta calculados. $\Delta w_{ij} = -\mu \nabla_w \xi = \mu \cdot \delta_j \cdot y_i(n)$ (delta peso)
5. Pasada de validación de 2, 3 y 4.
6. Vuelvo a 2 hasta satisfacer criterio de parada o máx. épocas

Éxito

Scanned with CamScanner

RBF-NN

redes neuronales con funciones de base radial

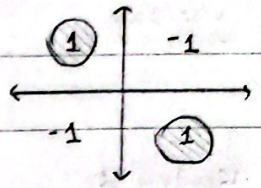


una capa radial → sin sesgo
con gaussianos
cada neurona tiene sus parámetros μ, σ.

en vez de las sismoideas, usamos funciones que sean como círculos, así formar regiones radiales.

$$y_k(\underline{x}_l) = \sum_{j=1}^{M} w_{kj}\,\phi_j(\underline{x}_l)$$

$$\phi_j(\underline{x}_l) = e^{\frac{-\|x_l - \mu_j\|^2}{2\sigma_j}}$$

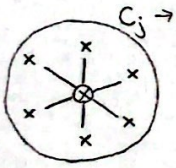


→ **Entrenamiento** → **mixto** → adaptación **no supervisada** de las RBF (↗ K-medias, SOM.)
→ adaptación **supervisada** de los pesos w_j de la capa lineal de salida.

} podemos emplear con una adaptación supervisada de los RBF

- **K-MEDIAS**

encontrar conjunto $p/$ cada neurona tal que cada conjunto sea lo más distinto posible de los demás y los patrones x_l dentro de cada C_j sean lo más parecidos entre ellos.



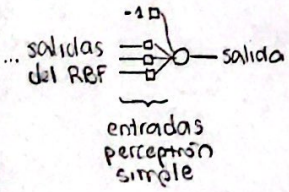
el centroide de la neurona j es la μ_j de su gaussiana.

- **por lotes:**
  1. Inicializo $\underline{x}_j$ aleatoriamente $p/$ cada C_j
  2. Promediamos los $\underline{x}_j$ de cada conjunto C_j para obtener centroide μ_j.
  3. Reasigno a cada $\underline{x}_j$ al conjunto cuyo centroide está más cerca.
  4. Volver a 2 hasta que no se realicen reasignaciones.

- **online:**
  1. Elijo K patrones aleatorios como centroides iniciales.
  2. Busco centroide con menor distancia a $c/$ patrón y se lo asigno.
  3. Actualizo coordenadas del centroide.
  4. Volver a 2 hasta no encontrar mejoras significativas en J.

- **Adaptación de pesos:** una vez entrenada la capa radial, la fijamos y obtenemos las salidas intermedias, entrenamos la capa lineal como un perceptrón simple:

$$y = W.\phi(x_l)$$



- **pseudo-inversa de $\phi(x_l)$** $\quad y = \underline{W}\,\underline{\Phi}(\underline{x}_l)$

$$\underline{W} = \phi^{-1}(\underline{x}_l).y$$

- **LMS (gradiente)** lo que ya conocemos, nos movemos en la dir. opuesta a $\nabla\xi$. →

$$e_k(n) = y_k(n) - d_k(n)$$

$$\xi(n) = \frac{1}{2}\sum_k e_k^2(n)$$

$$w_{kj}(n+1) = w_{kj}(n) - \eta\left(\underbrace{\sum_i w_{ki}(n).\phi_i(n)}_{y_k(n)} - d_k(n)\right).\phi_j(n)$$

$\underbrace{\qquad\qquad\qquad\qquad}_{e_k(n)}$

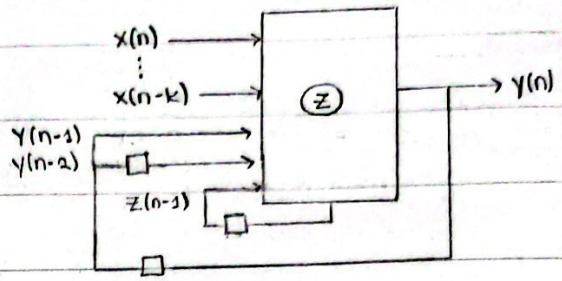
Éxito

Scanned with CamScanner

REDES NEURONALES DINÁMICAS

$y(n) = f(x(n), z_1(n), y_1(n))$

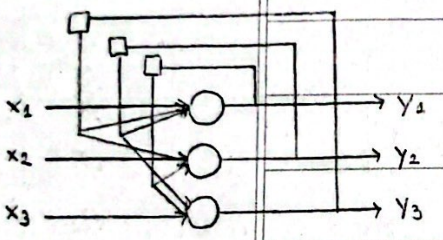
- $x(n)$: entradas de la red actuales y anteriores
- $z_1(n)$: salidas de capas ocultas anteriores
- $y_1(n)$: salidas de la red anteriores.



retroalimentación

Si despliego la red tomando c/instante de tiempo n-k como una capa, puedo resolver con la retropropagación
↳ pesos son los mismos pl las k capas!!

• Redes de Hopfield:



La retroalimentación no es propia: una neurona retroalimenta a las otras neuronas de la capa.
Misma cantidad de neuronas que entradas y salidas.
Entrenamiento no supervisado.
Cada neurona tiene un **disparo probabilístico**
↳ variable aleatoria se elige al azar a cual le toca disparar en c/ momento.

$$y_j(n) = \text{sgn}(x) = \text{sgn}\left(\sum_{i=1}^{N} w_{ji}\, y_i(n-1) - \theta_j\right) \dots \begin{cases} x>0: +1 \\ x=0: y_j(n-1) \\ x<0: -1 \end{cases}$$

↳ mantiene valor del instante anterior.

red binaria — pesos x entradas — sesgo

$w_{ji} = w_{ij}$ → realimentación igual peso en ambos sentidos.
$w_{ii} = 0$ → no realimento a una misma.

→ Entrenamiento:

Almacenamiento

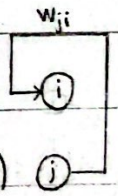
↗ los muestro UNA vez (no iterativo), no ajusto nada.

Datos limpios para entrenar, completos y sin ruido. Les hago un promedio, y listo.

Aprendizaje Hebbiano:

$$w_{ji} = \frac{1}{N} \cdot \sum_{k=1}^{P} x^*_{kj}\, x^*_{ki}, \quad P_{max} = \frac{N}{2 \cdot \ln(N)}$$

N → cant. neuronas
(límite de memorias fund. con 1% de error)



↳ detecta nivel de sincronización entre dos neuronas: → memoria de relación entre píxeles.
↳ peso grande si se activan/desactivan a la vez (positivo) o si siempre van al revés (neg.)
↳ peso chico si van siempre desincronizadas → sumatoria cancela los pos. y neg y queda un número chico cercano a cero.

→ Prueba:

Recuperación

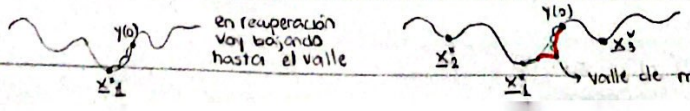
Datos incompletos y/o con ruido pl ver si recupera el patrón limpio.

Iterativo:

1. Elijo una neurona j^* al azar.
2. Calculo su salida $y_{j^*}(n) = \text{sgn}\left(\sum_{i=1}^{N} w_{ji}\, y_i(n-1)\right)$
3. Volver a 1 hasta no observar cambios en las y_j

Éxito
↳ red converge a la solución.
↳ red no converge por **estados espúreos** y **oscilaciones** → superé el Pmax.
↳ dos memorias fund. son muy parecidas.

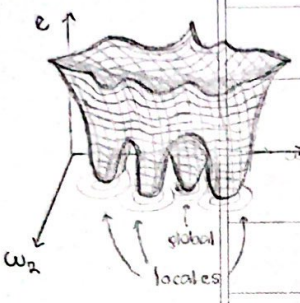
MEMORIAS FUNDAMENTALES
cantidad de patrones en el entrenamiento: imágenes, caras a detectar, que se almacenan.



Scanned with CamScanner

CAPACIDAD DE GENERALIZACIÓN

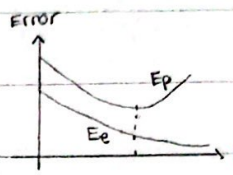
• SUPERFICIES DE ERROR

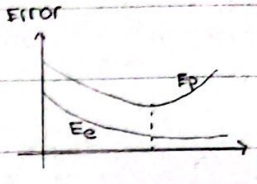


Si tiene muchos **mínimos locales** puede no converger o tardar porque no sabe encontrar el **mínimo global**.

para salir ↳ inicializo al azar muchas veces los pesos p/ que arranquen de distintos lugares y me quedo con el mínimo (para esto es solo error del entrenamiento)

• SOBRE-ENTRENAMIENTO:



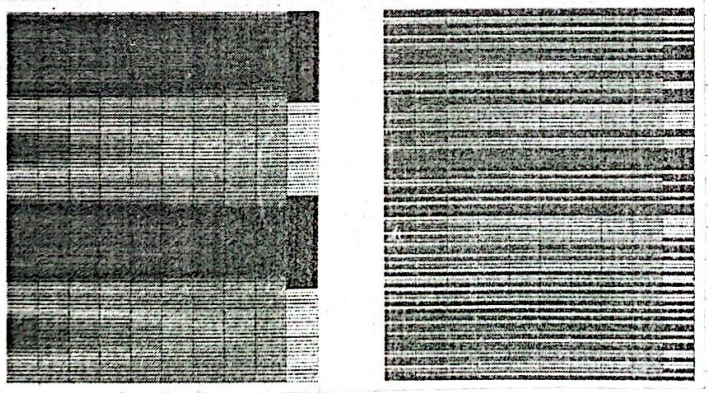


Si hago muchas épocas de entrenamiento, el modelo se ajusta mucho a los datos del entrenamiento. Se aprende datos específicos del dataset de entrenamiento.

Lo mismo si la arquitectura es muy compleja o grande.

↳ Lo evito con la validación (cuando Ep empieza a subir, corto)

• VALIDACIÓN CRUZADA

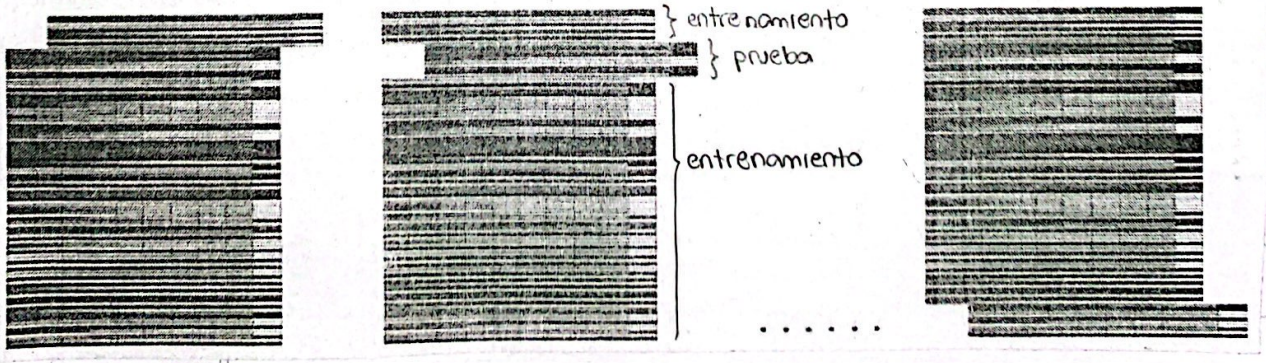


Primero reordeno los patrones al azar para no tomar solo patrones de un solo caso o clase para la prueba.

Para evitar tomar sólo casos difíciles o fáciles Repito este procedimiento tomando distintas particiones de los datos. (imagen abajo)

altern. p/ más particiones
- ↳ repitiendo patrones de particiones previas
- ↳ con reposición: un patrón puede aparecer dos veces en una misma partición

Calculo el error para cada partición y promedio p/ estimar desempeño: $\bar{e} = \frac{1}{N} \cdot \sum_{i=1}^{N} e_{pi}$ → ver la varianza p/ verificar los datos



leave-K-out: hago particiones de K patrones p/ prueba.

≠

K-fold: hago K particiones de patrones de prueba.

Éxito

Scanned with CamScanner

MEDIDAS DE DESEMPEÑO

• Matriz de confusión:

Me fijo en la diagonal los que pegó bien, los otros compas son los errores.

		Predicciones ⊕	Predicciones ⊖	
Clases correctas	⊕	$t_⊕$	$f_⊖$	$N_⊕ = t_⊕ + f_⊖$ → total casos positivos
	⊖	$f_⊕$	$t_⊖$	$N_⊖ = f_⊕ + t_⊖$ → total casos negativos
				$N = N_⊕ + N_⊖$ → total de casos a entrenar.

• Indices de Desempeño:

Sensitivity: capacidad de recuperar casos ⊕

$$s^+ = \frac{t_⊕}{t_⊕ + f_⊖} = \frac{t_⊕}{N_⊕}$$

Specificity: capacidad de recuperar casos ⊖

$$s^- = \frac{t_⊖}{t_⊖ + f_⊕} = \frac{t_⊖}{N_⊖}$$

} los uso juntos porque si uso solo uno, tengo sólo la info de los positivos (sens.) o los negativos (spec.)

capaz uno me da bien pero el otro está dando todo mal.

Precisión: cuantos de los pos/neg identificados son realmente pos/neg

$$p = \frac{t_⊕}{t_⊕ + f_⊕}$$

p/ caso negativo →

$$n = \frac{t_⊖}{t_⊖ + f_⊖}$$

} esta medida y su opuesto (info de neg) funcionan solos. son más completas.

Accuracy: } la más intuitiva y la más utilizada

$$a = \frac{t_⊕ + t_⊖}{N}$$

F_1 Score: media harmónica

$$F_1 = 2 \cdot \frac{s^+ p}{s^+ + p}$$

G-measure:

$$G = \sqrt{s^+ \cdot p}$$

• Métricas de clustering

Importante pl determinar parámetros y detectar errores en los datos o problemas del bias del algoritmo, o cosas q' no sabía q' habían en los datos.

INTERNAS reciben como entradas los datos y las soluciones del clústering

EXTERNAS comparan soluciones entre sí o con un golden standard

Cohesión media: varianza en los clústers respecto del centroide.

Separación: distancia entre los centroides w_i, w_j de distintos clústeres. puedo calcular la total haciendo todos los pares y promediando.

Matriz de convergencia:

sol.1 \ ∩	1 4 5	2 6 3	Σ
1 3 5	2	1	3
2 4 6	1	2	3
Σ	3	3	6

TP: (1,5), (2,6)

FP: (1,3), (5,3), (4,6), (4,2

FN: (1,4), (4,5), (2,3), (3,6)

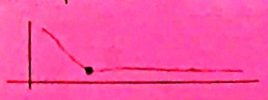
Medida combinada: suma la cohesión interna y divide por la separación

Fowlkes and Mallows:

$$FMI = \frac{TP}{\sqrt{(TP+FP)(TP+FN)}}$$

cuento pares de puntos iguales entre clusters (TP) los de s_1 y no s_2 (FP) y los de s_2 y no s_1 (FN)

K-óptimo: "elbow": calculo métricas para un rango de K y grafico la curva.



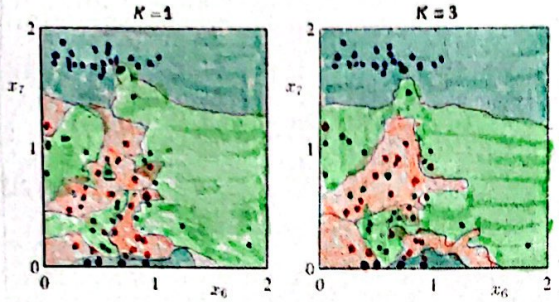
Rand Index: Mido el nro de pares en el mismo conjunto s_1 y s_2 (a) y el nro de pares q' están en diferentes (b).

$$RI = \frac{a+b}{C_2^{nsamples}}$$

Scanned with CamScanner

Aprendizaje Automático No Neuronal

- **K vecinos más cercanos:** dado un punto, lo clasifica según la clase que tiene los k vecinos más cercanos al punto.
  así, genera regiones de decisión para cada clase.

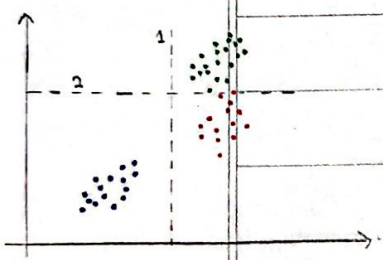


Ventajas:
- no aprende, solo revisa el dataset.
- no tiene parámetros pl ajustar o entrenar.
- con k apropiado es robusto al ruido.
- efectivo pl espacios muy cubiertos.

Desventajas:
- hay q' determinar el k → métricas.
- hay q' determinar la distancia a usar.
- costoso de calcular, muere pl muchos datos
- sensible a outliers. y clases desbalanceadas.

- **Árboles de decisión:** busca una dimensión y trata de separar los datos lo mejor que puede en esa, haciendo fronteras ortogonales a las dimensiones.

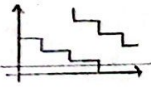
Para elegir qué dimensión divido primero, uso **medida de impureza** para evaluar qué dimensión tiene mayor impacto en la clasificación.



primero divide vertical pq' separa bien bien y dsps divide horizontal.

- **Impureza de clasificación:** $I_n^c = 1 - \max_i P(x \in w_i | n)$
  mide mejor proporción de clase en la región y lo resta de 1. Mientras mejor sea I_n^c, más pura la región.
- **Impureza de Gini:** $I_n^G = \sum_{i \neq j} P(x \in w_i | n) P(x \in w_j | n) = 1 - \sum_i P^2(x \in w_i | n)$
  probabilidad de que un ejemplo al azar se clasifique mal si le doy una clase aleatoria dentro de la distribución de clases de la región. baja → mejor.
- **Impureza de Entropía:** $I_n^H = -\sum_i P(x \in w_i | n) \log P(x \in w_i | n)$
  mide incertidumbre en la distr. de clases.
  alta: conjunto impuro
  baja: conjunto puro, predecible.

Ventajas:
- muy interpretables → cadena de if-then
- no requiere transformar los datos, sean categóricos o numéricos.
- baja costo computacional.
- no requiere tantos datos pl entrenar.
- robustos a outliers.



para ciertos problemas se va complicando pero se puede.

Desventajas:
- no son robustos a ruido en los datos
- fronteras ortogonales
- no escalan bien con complejidad.
- se sobreajustan con facilidad → modelan el ruido. pl evitar pongo máxima de profundidad.

- **Clasificadores Bayesianos Ingenuos:** son ingenuos porque asumen independencia entre las dimensiones o características. así pueden simplificar los cálculos del Teorema de Bayes.
  → calculo $P_i(w|x)$ pl cada clase y me quedo con la más alta.

Ventajas:
- datos categóricos o numéricos
- crece linealmente con cant. de dim.
- fácil de implementar
- rápido

Éxito

$$P(w|x) = \frac{P(x|w) \cdot P(w)}{P(x)}$$ (Teorema de Bayes)

$$P(x|w) = P(x_1|w) \cdot P(x_2|w) \dots P(x_n|w)$$
(simplificación por asumir independencia).

Desventajas:
- independencia no siempre se cumple.
- problema de estimación al entrenar.

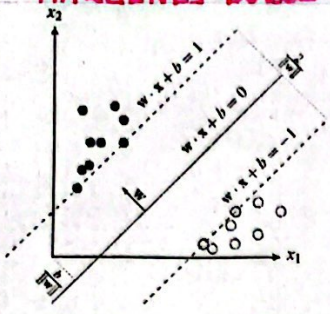
Scanned with CamScanner

• Máquinas con vectores de soporte:

encuentran un hiperplano de separación óptimo en un espacio de características para clasificar datos en dos o más categorías.

↳ trazo una línea, busco puntos más cercanos de ambas clases y trazo líneas paralelas a ésta, que pasen por los puntos más cercanos.

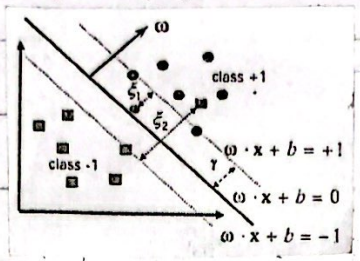
MÁRGENES DUROS



(problemas linealmente separables)

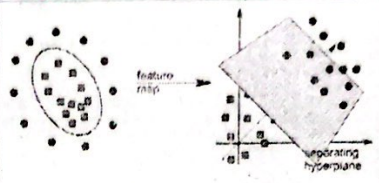
MÁRGENES BLANDOS

permite ciertos errores para mejorar hiperplanos.



(problemas no linealmente separables)

truco del kernel



transforma datos a otro espacio geométrico así puedo separar con un plano, usando el mismo algoritmo que antes.

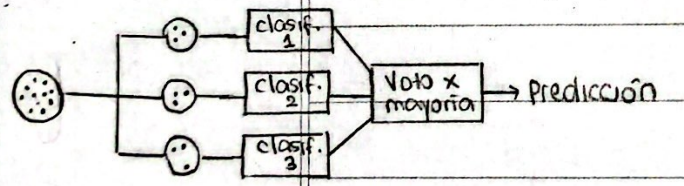
Ventajas:
- rápida convergencia en entrenamiento → calculo productos en espacio original, no $\mathbb{R}^{grande}$.
- funciona bien p/ muchas dimensiones.
- ~~funciona mejor p/ menos datos q' dimensiones.~~
- funciona bien p/ márgenes amplios en el espacio proyectado.

Desventajas:
- elegir el kernel e hiperparámetros
- problemas con grandes conjuntos de datos.
- no maneja bien solapamiento de clases.
- no es muy interpretable en cuanto a sus decisiones.

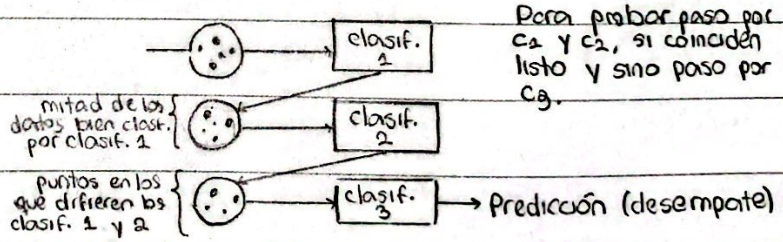
• Ensambles de clasificadores:

clasificador formado por clasificadores individuales, simples, para aprovechar su independencia entre sí.

Bagging toma subconj. de datos al azar p/ cada clasificador y después hago votación.



Boosting cada clasificador toma datos procesados por otro clasificador, no son al azar.



Para probar paso por c_1 y c_2, si coinciden listo y sino paso por c_3.

Ventajas:
- difícil sobreentrenar
- se adaptan al desbalance de clases.
- combinar muchos tipos de clasif. y datos.
- aprovechan pocos datos.
- tratan naturalmente los problemas multiclase.
- estables y robustos.

Desventajas:
- costo computacional y de memoria.
- difícil de interpretar y entender decisiones.
- resultados varían mucho según la elección de modelos y datos.

ADABoost (boosting adaptativo)

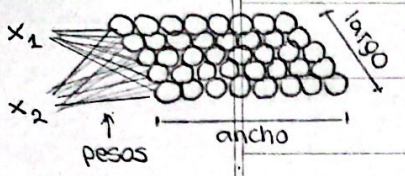
es secuencial pero le paso al próximo clasificador los casos difíciles que el anterior no pudo clasificar bien.

luego hago un ensamble ponderado, dándole más peso a los clasificadores de mejor desempeño.

Éxito

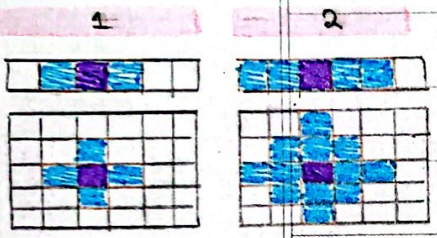
Scanned with CamScanner

MAPAS AUTOORGANIZATIVOS



Red neuronal de **competencia**: UNA neurona y su entorno se activan. Se aplica cuando no sé salidas y quiero detectar clases. Las entradas están conectadas a todas las neuronas. Si tengo n entradas, voy a tener pesos x ancho vectores de tamaño n para los pesos asociados a cada neurona. Además, tengo pesos de conexión o vecindad entre neuronas (grilla).

Radio de activación:

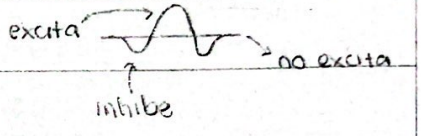


Funciones de excitación o inhibición lateral: ⇒ cuánto activo/inhibo las vecinas.

UNIFORME: si está en el radio, la actualizo igual que a la ganadora.

GAUSSIANA: si está en el radio, se actualiza con más peso mientras más cerca de la ganadora.

SOMBRERO MEXICANO:



→ Entrenamiento:

1. Inicializo pesos al azar $[-0.5, 0.5]$ o un patrón de entrada al azar.
2. selecciono neurona ganadora (la más cercana al patrón actual)
3. Ajusto los pesos:

$$\underline{w}_j(n+1) = \begin{cases} \underline{w}_j(n) + \eta(n) \cdot (\underline{x}(n) - \underline{w}_j(n)), & \text{si vecina} \\ \underline{w}_j(n), & \text{sino} \end{cases}$$

η: vel. de aprendizaje; $(\underline{x}(n) - \underline{w}_j(n))$: vector de error

4. Vuelvo a 2 hasta no ver cambios significativos, o máx. de épocas.

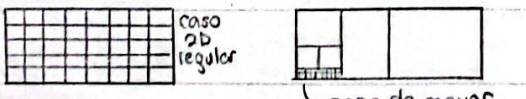
ETAPAS		
Ord. Global	Transic.	Ajuste Fino
• vecinos medio mapa • η grande • 500 a 1000 épocas	• vecinos se reducen lineal hasta 1. • η se reduce lineal hasta 1 • 1000 epocas	• 0 vecinos • η cte $[0.1, 0.01]$ • Hasta converger ~3000 epocas

Formación de mapas topológicos: Las neuronas cercanas en $\mathbb{R}^2$ representan patrones que están cerca entre sí en $\mathbb{R}^N$. Estoy logrando una proyección para representar fácilmente.

• LVQ1

variante inicial

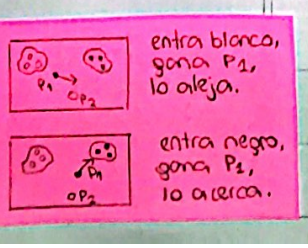
cuantización vectorial con aprendizaje → SOM lineal, supervisado, sin entorno. tomo una función y la cuantizo para poder digitalizarla.



como en cada actualización de pesos multiplico por todos los α previos, si $\alpha < 1$ olvida muy rápido. por eso empiezo con un α grande y voy disminuyendo, para equiparar pesos en el valor final.

→ Entrenamiento:

1. Inicialización aleatoria
2. Selecciono centroide más similar al patrón de entrada (menor distancia)
3. Adapto los pesos: si el prototipo (neurona) ganador es el prototipo de la clase, lo acerco al patrón que acaba de entrar. sino, lo aleja.
4. Vuelvo a 2 hasta satisfacer error de clasificación.



Scanned with CamScanner